High-Dimensional Feature Selection via Permutation Test and Bootstrap Stability Analysis

Renfei Wu, Shiqi Tian, Chengxi Li, Liwen Yin

2026-02-24

1. Data Simulation

(1). `simulate_mixture_data()`

This function generates a synthetic dataset for testing feature selection methods. It creates an $n \times p$ feature matrix X** and binary labels **Y**, where only `n_relevant` features are truly associated with Y — the rest are pure noise. The indices of the true relevant features are stored in **S**, which serves as ground truth for evaluation later (R Core Team 2025).

```r
sim <- simulate_mixture_data(
  n          = 200,
  p          = 50,
  n_relevant = 10,
  alpha      = 0.7,
  dist_G     = "normal",
  dist_H     = "normal",
  seed       = 123
)

dim(sim$X)   # 200 x 50
```

```
## [1] 200  50
```

```r
sort(sim$S)  # true relevant feature indices
```

```
##  [1]  1  6 14 17 26 27 29 42 45 46
```

```r
table(sim$Y) # class balance
```

```
##
##   0   1
## 100 100
```

```r
names(sim)
```

```
## [1] "X"      "Y"      "S"      "theta"  "phi"    "alpha"  "dist_G" "dist_H"
```

The dataset has 200 observations and 50 features, of which 10 are truly relevant. These indices in `sim$S` will be used as ground truth throughout the rest of this vignette.

2. Permutation Testing

(1).`stat_mean_diff()`

This function computes the absolute difference between the group means of a single feature. A larger value indicates a stronger difference between the two classes. which is,

$$\text{stat} = |\bar{x}_1 - \bar{x}_0|$$

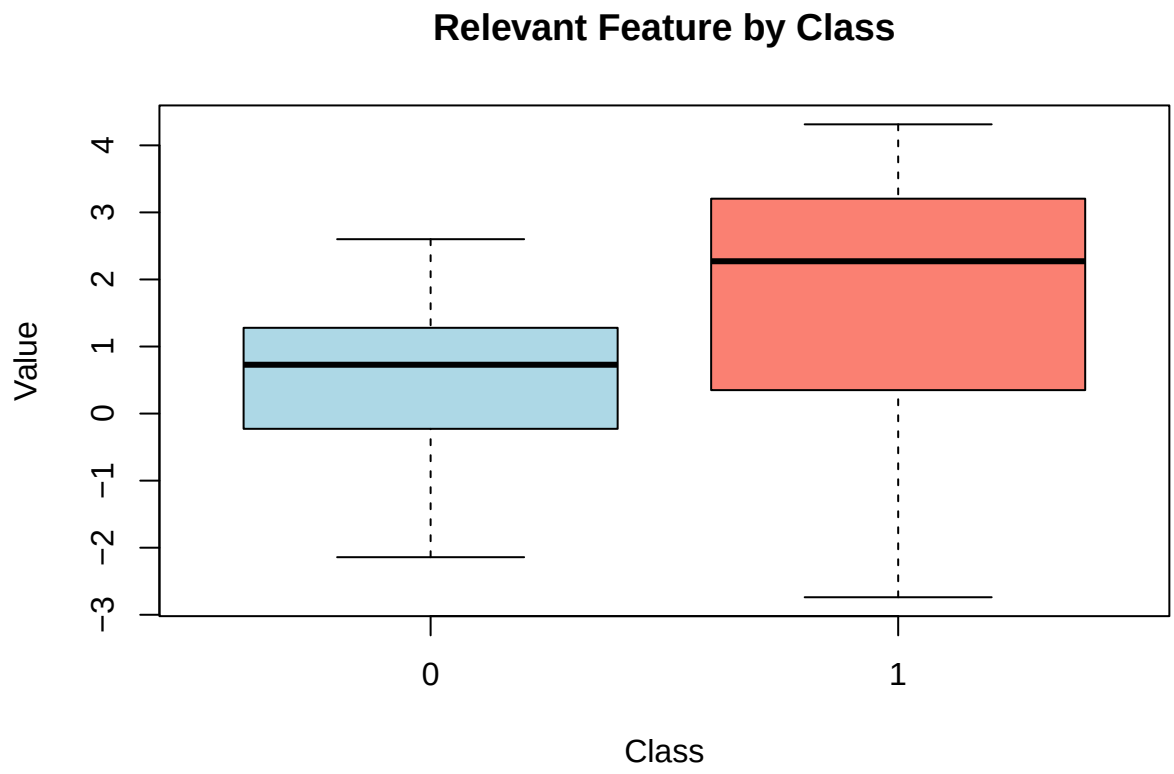
```
# get the most relevant feature
stats        <- sapply(sim$S, function(j) stat_mean_diff(sim$X[, j], sim$Y))
best_j       <- sim$S[which.max(stats)]

x_relevant   <- sim$X[, best_j]  # most strongly relevant feature
x_irrelevant <- sim$X[, setdiff(1:50, sim$S)[1]]   # returns all feature indices that are not contained

stat_mean_diff(x_relevant, sim$Y)
```

```
## [1] 1.313295
```

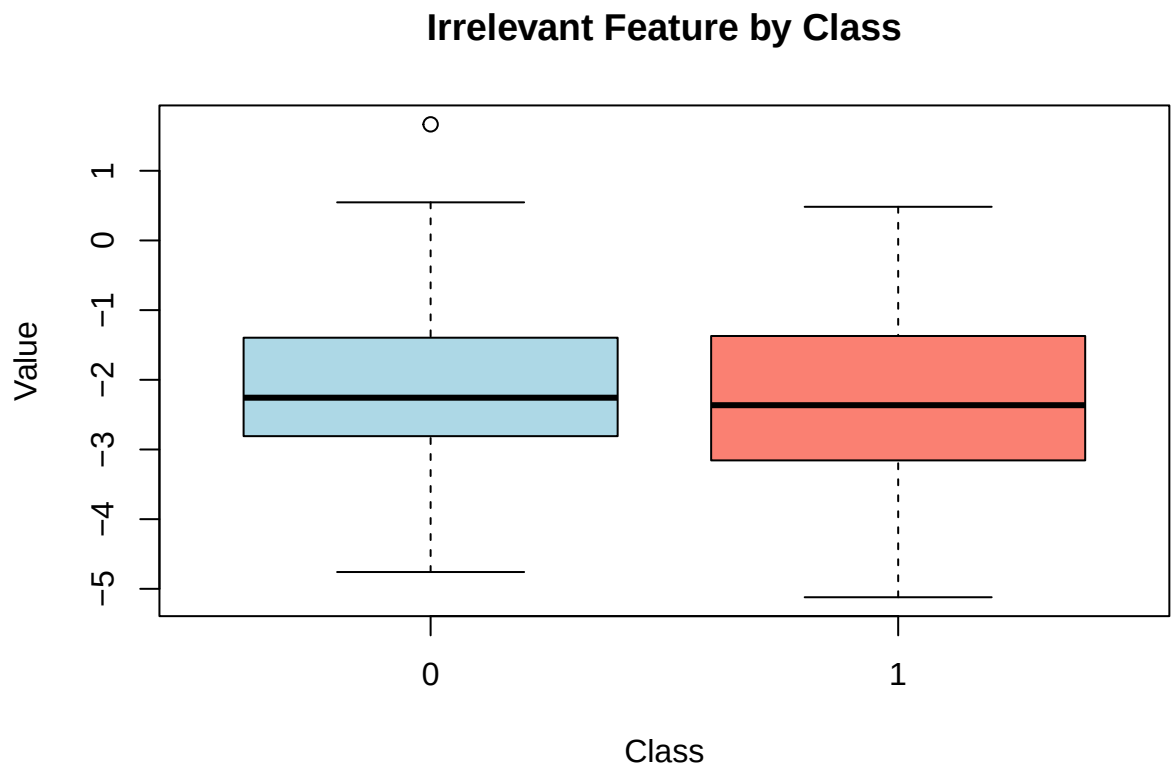
```
stat_mean_diff(x_irrelevant, sim$Y)
```

```
## [1] 0.1770272
```

```
# display plot

boxplot(x_relevant ~ sim$Y,
        main = "Relevant Feature by Class",
        xlab = "Class",
        ylab = "Value",
        col = c("lightblue", "salmon"))
```




```r
boxplot(x_irrelevant ~ sim$Y,
        main = "Irrelevant Feature by Class",
        xlab = "Class",
        ylab = "Value",
        col = c("lightblue", "salmon"))
```



The relevant feature produces a larger statistic than the irrelevant one, confirming that the function captures the signal correctly.

The boxplots show that the relevant feature exhibits a clear separation between the two classes, whereas the irrelevant feature shows substantial overlap. This visual difference aligns with the larger test statistic observed earlier.

(2). `stat_ks()`

This function computes the Kolmogorov-Smirnov statistic, which measures the maximum difference between the empirical distributions of a feature in class 0 and class 1. Unlike `stat_mean_diff()`, it captures differences in the overall shape of the distribution, not just the mean.

```
stat_ks(x_relevant, sim$Y)
```

```
## [1] 0.61
```

```
stat_ks(x_irrelevant, sim$Y)
```

```
## [1] 0.15
```

Again, the relevant feature produces a larger statistic, indicating a greater distributional difference between the two classes.

(3). `stat_cvm()`

This function computes the Cramér-von Mises statistic, which measures the overall difference between two empirical distributions by summing the squared differences at all observed points. It is more sensitive to differences across the whole distribution compared to `stat_ks()` (Good 2005).

```
stat_cvm(x_relevant, sim$Y)
```

```
## [1] 5.3593
```

```
stat_cvm(x_irrelevant, sim$Y)
```

```
## [1] 0.2157
```

As expected, the relevant feature yields a larger statistic than the irrelevant feature.

(4). `permutation_pvalue_one()`

This function computes a permutation p-value for a single feature. It repeatedly shuffles the class labels $\mathbf{Y}$ and recomputes the test statistic each time, building a null distribution. The p-value is the proportion of shuffles that produced a statistic as large as the observed one. A small p-value suggests the feature is truly associated with $\mathbf{Y}$.

```
set.seed(380)
permutation_pvalue_one(x_relevant, sim$Y,
                       stat_fun   = stat_mean_diff,
                       n_shuffles = 199)
```

```
## [1] 0.005
```

```
permutation_pvalue_one(x_irrelevant, sim$Y,
                       stat_fun   = stat_mean_diff,
                       n_shuffles = 199)
```

```
## [1] 0.29
```

The relevant feature gives a much smaller p-value than the irrelevant feature, indicating strong evidence of association with $\mathbf{Y}$.

(5). `permutation_pvalues()`

This function applies `permutation_pvalue_one()` to every column of $\mathbf{X}$, returning a vector of p-values of length p. This allows us to assess all features at once.

```
set.seed(380)
pvals <- permutation_pvalues(sim$X, sim$Y,
                             stat_fun = stat_mean_diff,
                             n_shuffles = 199)
length(pvals)
```

```
## [1] 50
```

```r
summary(pvals)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.0050  0.2375  0.5000  0.4818  0.7087  1.0000
```

The function returns one p-value per feature. Features with small p-values are candidates for selection.

(6). `select_features_permutation()`

This function selects features whose permutation p-value is at or below a chosen significance level `alpha_level`. It is the final step of the permutation testing pipeline.

```r
set.seed(380)
selected_perm <- select_features_permutation(
  sim$X, sim$Y,
  stat_fun    = stat_mean_diff,
  n_shuffles  = 199,
  alpha_level = 0.05
)

selected_perm  # selected feature indices
```

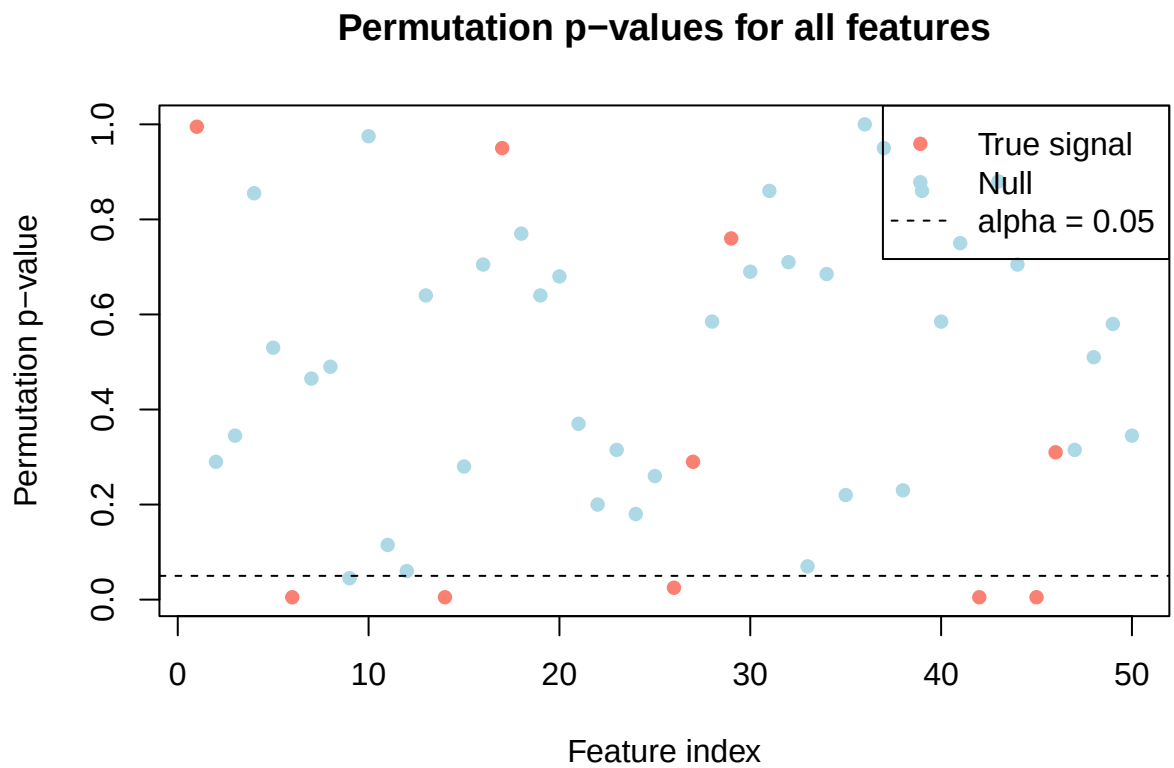
```
## [1]  6  9 14 26 42 45
```

```r
sim$S          # true relevant features for comparison
```

```
##  [1]  1  6 14 17 26 27 29 42 45 46
```

display the p-values for all 50 features:

```r
plot(pvals,
     xlab = "Feature index",
     ylab = "Permutation p-value",
     main = "Permutation p-values for all features",
     pch  = 16,
     col  = ifelse(1:50 %in% sim$S, "salmon", "lightblue"))
abline(h = 0.05, lty = 2)
legend("topright",
       legend = c("True signal", "Null", "alpha = 0.05"),
       col    = c("salmon", "lightblue", "black"),
       pch    = c(16, 16, NA),
       lty    = c(NA, NA, 2))
```



Features below the dashed line are selected. True signal features are shown in red.

3. Bootstrap Stability

(1). `bootstrap_selection_frequency()`

A single permutation test can vary depending on the sample. This function runs the feature selection procedure across B bootstrap resamples and records how often each feature is selected. A higher frequency indicates a more stable and reliable feature (Efron and Tibshirani 1994).

```
selector <- function(X, Y) {
  select_features_permutation(X, Y,
                              stat_fun    = stat_mean_diff,
                              n_shuffles  = 99,
                              alpha_level = 0.05)
}

set.seed(380)
freq <- bootstrap_selection_frequency(
  sim$X, sim$Y,
  select_fun = selector,
  B          = 50,
  seed       = 380
)

length(freq)
```

```
## [1] 50
```

```r
round(freq, 2)
```

```
##  [1] 0.04 0.26 0.16 0.02 0.10 0.86 0.06 0.08 0.60 0.00 0.20 0.66 0.06 1.00 0.16
## [16] 0.06 0.00 0.08 0.06 0.08 0.22 0.18 0.16 0.30 0.22 0.78 0.12 0.16 0.08 0.08
## [31] 0.08 0.08 0.50 0.14 0.26 0.02 0.06 0.26 0.00 0.08 0.12 1.00 0.06 0.08 1.00
## [46] 0.22 0.18 0.14 0.00 0.12
```

Features in the true signal set tend to have higher selection frequencies than noise features.

(2). `select_features_bootstrap()`

This function selects features whose bootstrap selection frequency is at or above a `threshold`. Only features that are consistently selected across bootstrap resamples are retained, making the selection more stable. It also returns the full frequency vector as an attribute.

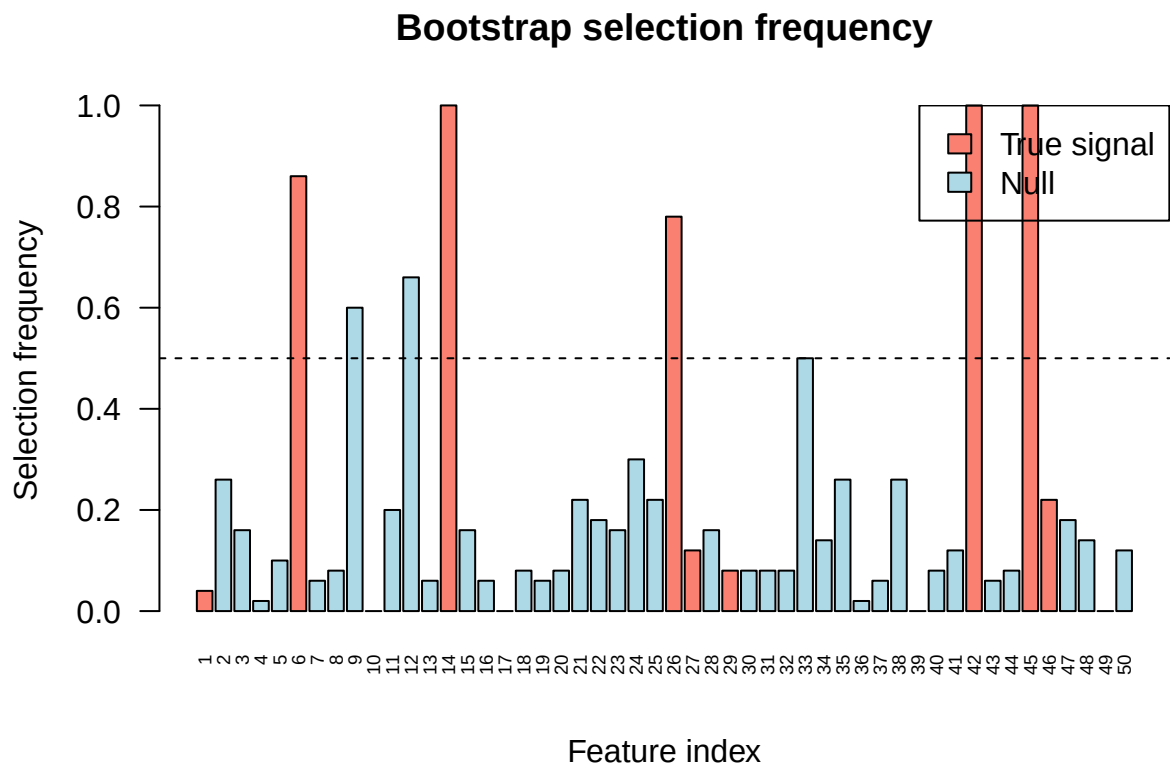
```r
set.seed(380)
selected_boot <- select_features_bootstrap(
  sim$X, sim$Y,
  select_fun = selector,
  B          = 50,
  threshold  = 0.5,
  seed       = 380
)

selected_boot
```

```
## [1]  6  9 12 14 26 33 42 45
## attr(,"frequency")
##  [1] 0.04 0.26 0.16 0.02 0.10 0.86 0.06 0.08 0.60 0.00 0.20 0.66 0.06 1.00 0.16
## [16] 0.06 0.00 0.08 0.06 0.08 0.22 0.18 0.16 0.30 0.22 0.78 0.12 0.16 0.08 0.08
## [31] 0.08 0.08 0.50 0.14 0.26 0.02 0.06 0.26 0.00 0.08 0.12 1.00 0.06 0.08 1.00
## [46] 0.22 0.18 0.14 0.00 0.12
```

```r
attr(selected_boot, "frequency")[selected_boot]
```

```
## [1] 0.86 0.60 0.66 1.00 0.78 0.50 1.00 1.00
```

```r
barplot(attr(selected_boot, "frequency"),
        names.arg = 1:50,
        col       = ifelse(1:50 %in% sim$S, "salmon", "lightblue"),
        xlab      = "Feature index",
        ylab      = "Selection frequency",
        main      = "Bootstrap selection frequency",
        las       = 2,
        cex.names = 0.6)
abline(h = 0.5, lty = 2)
legend("topright",
       legend = c("True signal", "Null"),
       fill   = c("salmon", "lightblue"))
```



8 features above the dashed line are selected. True signal features are shown in red.

4. Evaluation

(1). `evaluate_precision_recall()`

This function compares the selected features against the true relevant set **S**. Precision measures how many selected features are truly relevant. Recall measures how many true relevant features were found.

```
pr <- evaluate_precision_recall(
  S_truth = sim$S,
  S_hat   = selected_boot
)

pr$precision
```

```
## [1] 0.625
```

```
pr$recall
```

```
## [1] 0.5
```

A higher precision means fewer false positives were selected. A higher recall means more true relevant features were found.

(2). `f1_score()`

The F1 score combines precision and recall into a single number. Both must be high for a good F1 score (Voss 2013).

$$F1 = \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

```r
f1_score(pr$precision, pr$recall)
```

```
## [1] 0.5555556
```

An F1 score closer to 1 indicates better overall feature selection performance.

Generative AI Statement

Generative AI (Cursor) was used in this project to assist with two aspects: (1) generating the plotting code for the visualizations in the vignette, and (2) refining the written descriptions of each function to improve clarity and structure. All code and descriptions were reviewed and verified by the author.

Bibliography

Efron, Bradley, and Robert J Tibshirani. 1994. *An Introduction to the Bootstrap*. Chapman; Hall/CRC.

Good, Phillip. 2005. *Permutation, Parametric and Bootstrap Tests of Hypotheses*. Springer.

R Core Team. 2025. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. https://www.R-project.org/.

Voss, Jochen. 2013. *An Introduction to Statistical Computing: A Simulation-Based Approach*. John Wiley & Sons.